



SIMD vs. SIMT

- SIMD: one thread + parallel OPs
- SIMT: many threads + scalar OPs

好处

T: Thread

- Address scattering
- Thread divergence 线程分支
- Number of instructions in ISA

C code	<pre>void vect_mult(int n, double *a, double *b, double *c) { for (int i = 0; i < n; ++i) a[i] = b[i] * c[i]; }</pre>
ARM (NEON)	<pre>void vect_mult(int n, uint32_t *a, uint32_t *b, uint32_t *c) { for(i = 0; i < n; i += 4) { uint32x4_t a4 = vld1q_u32(a+i); uint32x4_t b4 = vld1q_u32(b+i); uint32x4_t c4 = vmulq_u32(a4, b4); vst1q_u32(c+i, c4); } }</pre> 一次操作4个元素 从程序上还是一个 thread, 但是一次处 理多个元素
CUDA	<pre>__host__ int nblocks = (n+511)/512 // 512 threads per Thread Block vect_mult<<<nblocks, 512>>>(n, a, b, c); __device__ void vect_mult(int n, double *a, double *b, double *c) { int i = blockIdx.x*blockDim.x + threadIdx.x; if (i < n) a[i] = b[i] * c[i]; }</pre>

线程发散例如

Hierarchical thread model, **thread block**

- Cooperation between threads,
e.g. reduction mapped on large-scaled hardware,
reduce the communication cost
- Flexible number of threads for higher TLP,
thread blocks > # SMs
- Flexible scheduling on blocks
less aware of number of SMs in different GPUs

感知不同GPU中SM数量的无块灵活调度

比如比较低端的GPU中，如果SM比较少，可以分时复用，执行多个thread blocks

When hardware continue to grow in scale, **thread block cluster** introduced in Hopper

当硬件规模持续增长时，Hopper引入了线程块集群



Cluster内，不同SM之间也有快速通信通道



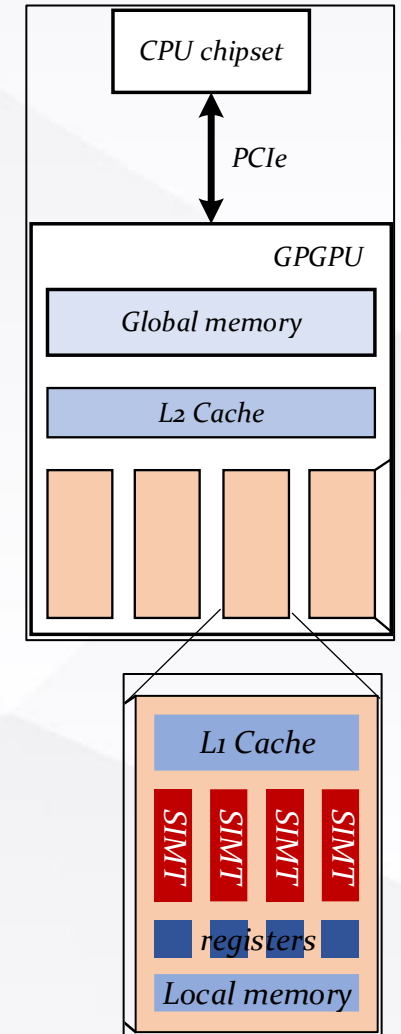
为了提供shared

memory

如果缺少这个层次，
只是grid

→thread，则线程间
交换数据需要借助

global memory



Programmability
hardware complexity
scalability

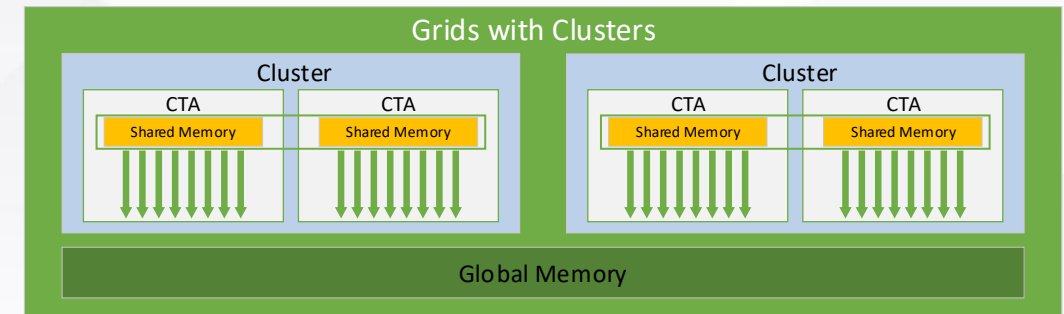
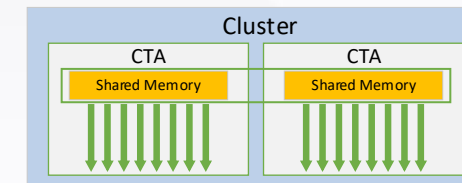
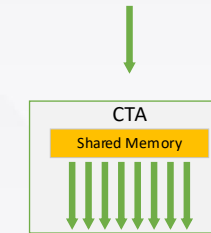
Heterogeneous Compute with GPU Thread Model

Operation	CUDA Cores / SFU / Tensor Cores
Thread	Streaming Processor (SP)
Thread block	Streaming Multiprocessor (SM)

Kernel (grid) — Whole GPU

Application — Host-device heterogeneous computing

$+/-/≤/≥/...$



CPU/memory

OpenMP



CUDA



- Designed for parallel computing with high performance
- Provide high level abstraction aligned with hardware

- Target at CPU
- Handle 10~100 threads
- No hierarchical programming model, easy to manage, but not well applicable to thousands of threads
没有分层编程模型，易于管理，但不适用于数千个线程
- Global sync

- Target at GPU
- Handle 1000~10000 threads
- Hierarchical programming model like: Grid, Cluster, Thread Block, Thread
- Provide sync operations at different levels with different granularity

提供不同级别、不同粒度的同步操作。

不需要在global层级上做很costly的syn，效率比较高